

UNIT IV

4.0 INTRODUCTION

Pointers are one of the derived types in C. One of the powerful tool and easy to use once they are mastered.

Some of the advantages of pointers are listed below:

- ❖ A pointer enables us to access a variable that is defined outside the function.
- ❖ Pointers are more efficient in handling the data tables.
- ❖ Pointers reduce the length and complexity of a program.
- ❖ The use of a pointer array to character strings save data storage space in memory.

The real power of C lies in the proper use of pointers.

4.1 POINTER CONCEPTS

The basic data types in C are int, float, char double and void. Pointer is a special data type which is derived from these basic data types.

There are three concepts associated with the pointers are,

- ❖ Pointer Constants
- ❖ Pointer Values
- ❖ Pointer Variables

4.1.1 POINTER CONSTANT

- ❖ As we know, computers use their memory for storing the instructions of a program, as well as the values of the variables that are associated with it.
- ❖ The computer's memory is a sequential collection of 'storage cells'.
- ❖ Each cell can hold one byte of information, has a unique number associated with it called as 'address'.
- ❖ The computer addresses are numbered consecutively, starting from zero. The last address depends on the memory size.

- ❖ Let us assume the size of the memory is 64K then,

The total memory locations = 64K
 = 64 * 1K
 = 64 * 1024 bytes
 = 65536 bytes (locations)

- ❖ So, here the last address is 65535(started with 0).
- ❖ Physically they are divided into even bank and odd bank.
- ❖ Even bank is set of memory locations with even addresses. Like 0, 2, 4, 6.....65534.
- ❖ Odd bank is set of memory locations with odd addresses. Like 1, 3, 565535.

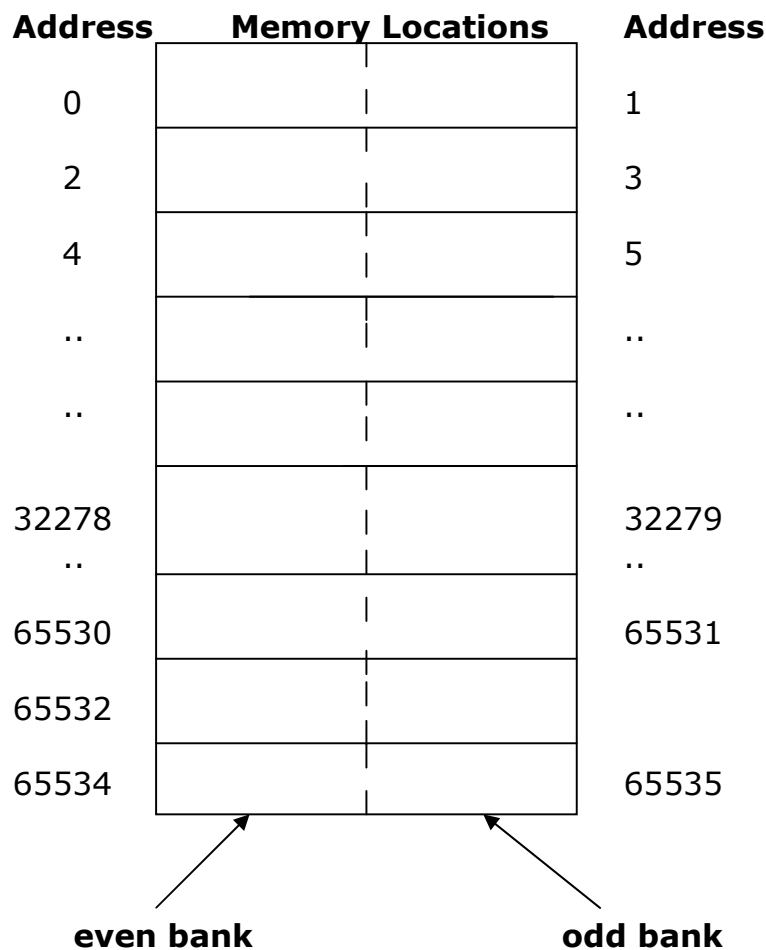


Figure: 4.1 Memory Organization.

- ❖ These memory addresses are called pointer constants.

- ❖ We cannot change them, but we can only use them to store data values.
- ❖ For example, in the above memory organization, the addresses ranging from 0 to 65535 are known as pointer constants.
- ❖ Remember one thing, the address of a memory location is a pointer constant and cannot be changed .

4.1.2 POINTER VALUE

Whenever we declare a variable, the system allocates , an appropriate location to hold the value of the variable somewhere in the memory,,

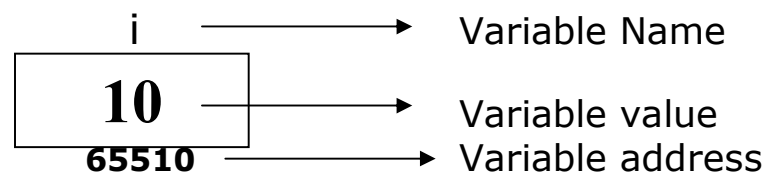
Consider the following declaration,

int i=10;

This declaration tells the C compiler to perform the following activities:

- ❖ Reserve **space** in the memory to hold the integer value.
- ❖ Associate the name **i** with this memory location.
- ❖ Store the value **10** at this location.

We can represent i's location in the memory by the following memory map:



Pointer Values

- ❖ Memory is divided into number of storage cells called locations.
- ❖ Out of these the addresses, the system assigns some addresses of the memory locations to the variables.
- ❖ These memory locations assigned to the variables by the system are called **pointer values**.
- ❖ For example, the address 65510 which is assigned to the variable i is a pointer value.

The & Operator

- ❖ The address of the variable cannot be accessed directly. The address can be obtained by using address operator(**&**) in C language.
- ❖ The address operator can be used with any variable that can be placed on the left side of an assignment operator.
- ❖ The format specifier of address is %u(unsigned integer),the reason is addresses are always positive values. We can also use %x to know the address of a variable.
- ❖ Example, to know the address of variable n, just use &n.

Note: Constants, expressions, and array name cannot be placed on the left side of the assignment and hence accessing address is invalid for constants, array names and expressions.

The following are illegal use of address Operator.

&125 (Pointing at constant)

int a[10];
&a (pointing to array name)

&(x+y) (pointing at expressions)

4.1.3 POINTER VARIABLE

- ❖ A variable Which holds the address of some other variable is called pointer variable.
- ❖ A pointer variable should contain always the address only.

The * Operator

- ❖ It is called as '**Value at address**' operator. It returns the value stored at a particular address.
- ❖ It is also Known as **Indirection** or **Dereferencing Operator**

4.2 ACCESSING A VARIABLE THROUGH POINTER

For accessing the variables through pointers, the following sequence of operations have to be performed ,to use pointers.

1. Declare an ordinary variable.
2. Declare a pointer variable.
3. Initialize a pointer variable(Provide link between pointer variable and ordinary variable).
4. Access the value of a variable using pointer variable.

We already familiar with the declaration and initialization of variable. Now we will discuss the remaining here.

Declaring a pointer variable

In C , every variable must be declared before they are used. Since the pointer variables contain address that belongs to a separate data type, they must be declared as pointers before we use them.

The syntax for declaring a pointer variable is as follows,

```
data type    *ptr_name;
```

This tells the compiler three things about the variable ptr_name.

1. The asterisk(*) tells that the variable ptr_name is a pointer variable.
2. ptr_name needs a memory location.
3. ptr_name points to a variable of type data type.

For example,

```
int *pi;
```

declares the variable p as a pointer variable that points to an integer data type. Remember that the type **int** refers to the data type of the variable being pointed by **pi**.

Initializing Pointers

- ❖ Once a pointer variable has been declared, it can be made to point to a variable using statement such as

```
ptr_name=&var;
```

- ❖ Which cause **ptr_name** to point to **var**. Now **ptr_name** contains the address of **var**. This is known as pointer initialization.
- ❖ Before a pointer is initialized it should not be used.

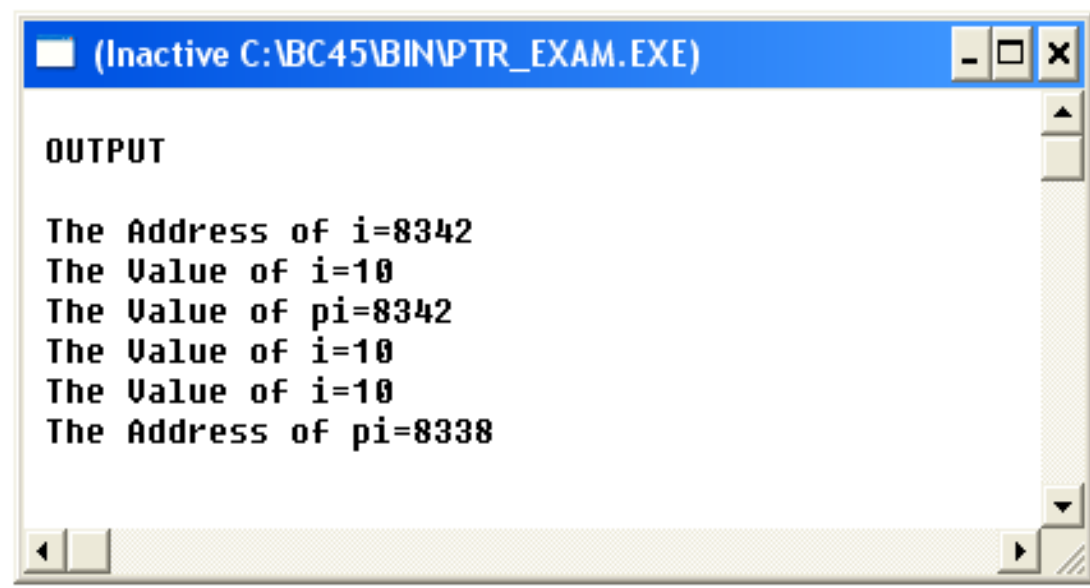
Access the value of a variable using pointer variable

Once a pointer variable has been assigned the address of a variable, we can access the value of a variable using the pointer. This is done by using the indirection operator(*).

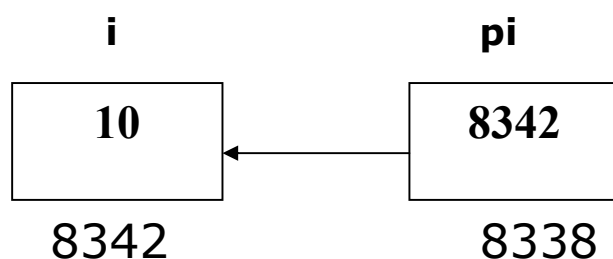
*ptr_name

Example1

```
// C Program illustrating accessing of variables using pointers
#include<stdio.h>
int main()
{
    int i=10;
    int *pi;
    pi=&i;
    printf("\n OUTPUT\n");
    printf("\n The Address of i=%u",&i);
    printf("\n The Value of i=%d",i);
    printf("\n The Value of pi=%u",pi);
    printf("\n The Value of i=%d",&i);
    printf("\n The Value of i=%d",*pi);
    printf("\n The Address of pi=%u",&pi);
    return 0;
}
```



The above program illustrates how to access the variable using pointers. After finding the first statement `i=10`, the compiler creates a variable `i` with a value of 10 at a memory location. Then coming to line 2 and 3 a pointer variable `pi` is created and initialized with the address of the `i` variable. then the compiler automatically provides a link between these two variables as follows.



Note: Pointer variable always points to a address of the another variable .Following statements are not valid with respect to pointers.

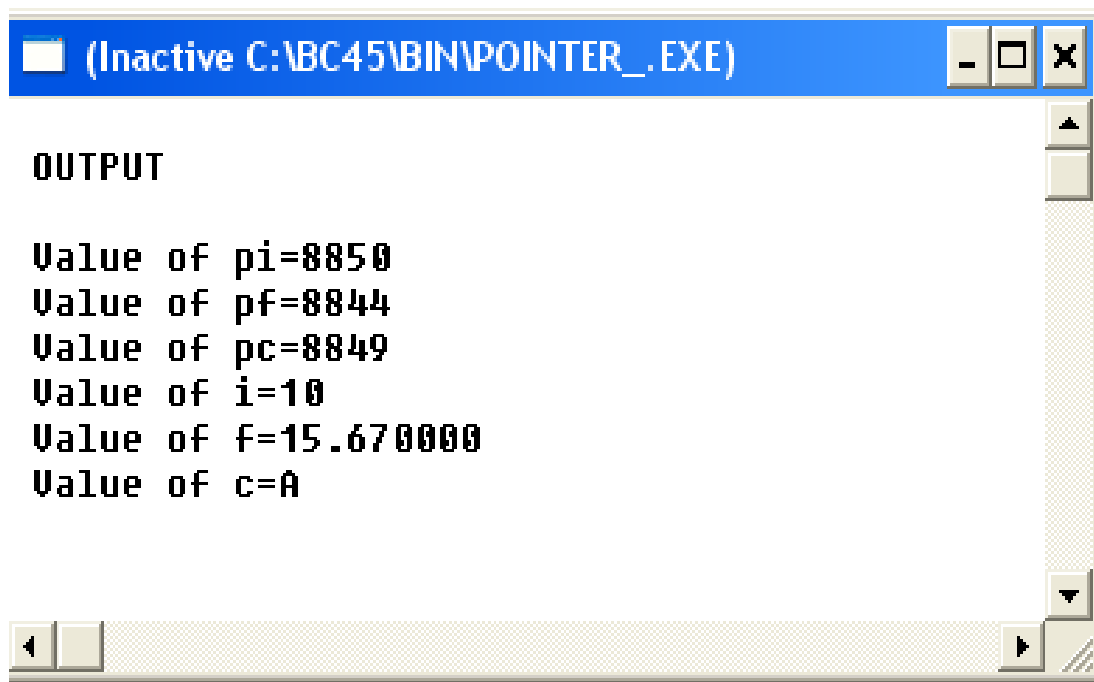
```

int i=10, k, *pi=&i;
k=pi; // pointer value cannot be accessed by integer
pi=65506(constant); // we cannot directly assign a value to a pointer
variable
  
```

Example2

The following code illustrates how to declare int ,char and float pointers. Here we have declared three variables of type int, float and char ,also three pointer variables points to int, float and char. Remember here pf points to the value of type float but its type is unsigned integer only.

```
#include<stdio.h>
int main()
{
    int i=10;
    char c='A';
    float f=15.67;
    int *pi; /*Pointer to an integer */
    float *pf; /*pointer to a float number */
    char *pc; /*pointer to a character; */
    pi=&i;
    pf=&f; /*initialization of pointer variables */
    pc=&c;
    printf("\n OUTPUT \n");
    printf("\n Value of pi=%u",pi);
    printf("\n Value of pf=%u",pf);
    printf("\n Value of pc=%u",pc);
    printf("\n Value of i=%d",*pi);
    printf("\n Value of f=%f",*pf);
    printf("\n Value of c=%c",*pc);
    return 0;
}
```

Declaration versus Redirection:

- ❖ When an asterisk is used for declaration, it is associated with a type.
- ❖ Example:

```
int* pa;  
int* pb;
```

- ❖ On the other hand, we also use the asterisk for redirection. When used for redirection, the asterisk is an operator that redirects the operation from the pointer variable to a data variable.
- ❖ Example:

```
Sum = *pa + *pb;
```

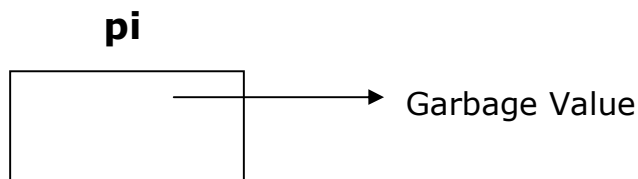
Dangling Pointers

A pointer variable should contain a valid address. A pointer variable which does not contain a valid address is called dangling pointer.

For example, consider the following declaration,

```
int *pi;
```

This declaration indicates that pi is a pointer variable and the corresponding memory location should contain address of an integer variable. But , the declaration will not initialize the memory location and memory contains garbage value as shown in below.



Note: We cannot use a pointer variable to the register variable. The reason is that, user does not know the address of the register variable. So we are not able to use pointer variable on register variables.

4.3 POINTER ARITHMETIC

The following operations can be performed on a pointer:

- ❖ Addition of a number to a pointer. Pointer can be incremented to point to the next locations.

Example:

```
int i=4 ,pi=&i; //(assume address of i=1000)
float j,*pj=&j;// (assume address of j=2000)
pi = pi + 1; // here pi incremented by (1*data type times)
pi = pi + 9; // pi = 1000 + (9*2) → 1018 address
pj = pj + 3; // pj=1018+(3*4)→1030 address
```

- ❖ Subtraction of a number from a pointer. Pointer can be decremented to point to the earlier locations.

Example:

```
int i=4,*pi=&i; //assume address of i =1000)
char c, *pc=&c; // assume address of c = 2000
double d, *pd=&d; // assume address of d=3000
pi = pi-2; /* pi=1000-(2*2)=996 address */
pc = pc-5; /* pc=2000-(5*1)=1985 address
pd = pd-6; /* pd=3000-(6*8)=2952 address */
```

- ❖ Pointer variables may be subtracted from one another. This is helpful while finding array boundaries. Be careful while performing subtraction of two pointers.
- ❖ Pointer variables can be used in comparisons, but usually only in a comparison to **NULL**.
- ❖ We can also use increment/decrement operators with pointers this is performed same as adding/subtraction of integer to/from pointer.

The following operations cannot be performed on pointers.

- ❖ Addition of two pointers.
- ❖ Multiplication of a pointer with a constant, two pointers.
- ❖ Division of a pointer with a constant, two pointers.

POINTER EXPRESSIONS

Like other variables, pointer variables can be used in expressions. For example, if p1 and p2 are two valid pointers, then the following statements are valid.

```
a = *p1 + *p2;
sum = sum + *p1;
z = 10 / *p2;
f = *p1 * i;
```

Note: be careful while writing pointer expressions. The expression ***p++** will result in the increment of the address of p by data type times and points to the new value. Whereas the expression **(*p)++** will increment the value at the address. If you are not properly coded you will get some unwanted result.

NULL Pointer

- ❖ If wish to have a pointer that points to "nowhere", should make this explicit by assigning it to NULL.
- ❖ If it is too early in the code to assign a value to a pointer, then it is better to assign NULL (i.e., \0 or 0).

```
double *pval1 = NULL;
double *pval2 = 0;
```

- ❖ The integer constants 0 and 0L are valid alternatives to NULL, but the symbolic constant is (arguably) more readable.
- ❖ A NULL pointer is defined as a special pointer.
- ❖ It can be used along with memory management functions.

4.4 POINTERS TO POINTERS

- ❖ It is possible to make a pointer to point to another pointer variable. But the pointer must be of a type that allows it to point to a pointer.
- ❖ A variable which contains the address of a pointer variable is known as **pointer to** pointer.
- ❖ Its major application is in referring the elements of the two dimensional array.
- ❖ Syntax for declaring pointer to pointer,

```
data type **ptr_ptr;
```

- ❖ This declaration tells compiler to allocate a memory for the variable **ptr_ptr** in which address of a **pointer variable** which points to value of type data type can be stored.
- ❖ Syntax for initialization

```
ptr_ptr=&ptr_name;
```

- ❖ This initialization tells the compiler that now ptr_ptr points to the address of a pointer variable.
- ❖ Accessing the element value,

```
**ptr_ptr;
```

- ❖ It is equalent to `*(&ptr_name);`

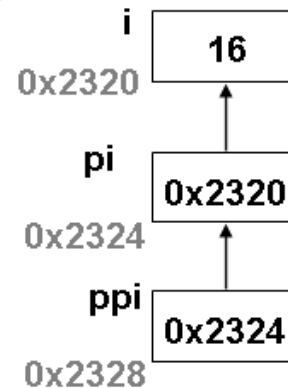
Example

```
#include <stdio.h>
int main(void)
{
    int i = 16;
    int *pi = &i;
    int **ppi;

    ppi = &pi;
    printf("%i", **ppi);

    return 0;
}
```

pp is a "pointer to" a
"pointer to an int"



The above program illustrates the use of pointers to pointers. Here, using two indirection operators the data item 16 can be accessed (i.e., *ppi refers to pi and **ppi refers to i).

4.5 POINTER COMPATIBILITY

- ❖ We should not store the address of a data variable of one type into a pointer variable of another type.
- ❖ During assigning we should see that the type of data variable and type of the pointer variable should be same or compatible.
- ❖ Other wise it will result in unwanted output.
- ❖ The following program segment is wrong,

```
int i=10;
```

```
float *pf;
```

```
pf=&i; // data variable is integer and pointer  
       variable is float
```

It is possible to use incompatible pointer types while assigning with type casting pointer.

Casting pointers

When assigning a memory address of a variable of one type to a pointer that points to another type it is best to use the cast operator to indicate the cast is intentional (this will remove the warning).

Example:

```
int V = 101;
float *P = (float *) &V; /* Casts int address to float * */
```

Removes warning, but is still a somewhat unsafe thing to do.

Void Pointer

- ❖ A pointer to void is a generic type that is not associated with a reference type.
- ❖ It is neither the address of a character nor an integer, nor a float nor any other type.
- ❖ It is compatible for assignment purposes only with all other pointer types.
- ❖ A pointer of any reference type can be assigned to a pointer to void type.
- ❖ A pointer to void type can be assigned to a pointer of any reference type.
- ❖ Certain library functions return void * results.
- ❖ No cast is needed to assign an address to a void * or from a void * to another pointer type.
- ❖ Where as a pointer to void can not be dereferenced unless it is cast.

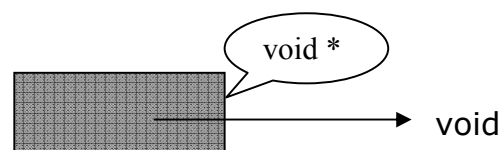


Figure 4.2 pointer to void

❖ Example:

```
int V = 101;
float f=98.45;
void *G = &V; /* No warning */
printf ("%d",*((int*)G)); /* Now it will display 101
float *P = G; /* No warning, still not safe */
printf ("%f",*((float*)G)); /* Now it will display 98.45
```

4.6 POINTERS AND FUNCTIONS

- ❖ Pointers can be used to pass addresses of variables to called functions, thus allowing the called function to alter the values stored there.
- ❖ We looked earlier at a swap function that did not change the values stored in the main program because only the values were passed to the function swap.
- ❖ This is known as "call by value".
- ❖ Here we are going to discuss how to pass the address.

Call by Reference

Instead of passing the values of the variables to the called function, we pass their addresses, so that the called function can change the values stored in the calling routine. This is known as "call by reference", since we are referencing the variables.

Here the addresses of actual arguments in the calling function are copied into formal arguments of the called function. Here The formal parameters should be declared as pointer variables to store the address.

The following shows the swap function modified from a "call by value" to a "call by reference". Note that the values are now swapped when the control is returned to main function.

#include <stdio.h>	void swap (int * pa , int * pb)
void swap (int * pa , int * pb);	{
int main ()	int temp ;
{	temp = * pa ; * pa = * pb ; * pb = temp ;
int a = 5, b = 6;	printf ("a=%d b=%d\n", * pa , * pb);
printf("a=%d b=%d\n", a , b) ;	}
swap (& a , & b) ;	<i>Results:</i>
printf("a=%d b=%d\n", a , b) ;	a=5 b=6
return 0 ;	a=6 b=5
}	a=6 b=5

Observe the following points when the program is executed,

- ❖ The address of actual parameters a and b are copied into formal parameters pa and pb.
- ❖ In the function header of swap (), the variables a and b are declared as pointer variables.
- ❖ The values of a and b accessed and changed using pointer variables pa and pb.

Call by Value	Call by Reference
<i>When Function is called the values of variables are passed.</i>	When a function is called address of variables is passed.
<i>Formal parameters contain the value of actual parameters.</i>	Formal parameters contain the address of actual parameters.
<i>Change of formal parameters in the function will not affect the actual parameters in the calling function</i>	The actual parameters are changed since the formal parameters indirectly manipulate the actual parameters
<i>Execution is slower since all the values have to be copied into formal parameters.</i>	Execution is faster since only addresses are copied.

Table: 4.1 Difference between Call by Value and Call by Reference

4.6.1 FUNCTION RETURNING POINTERS

- ❖ The way function return an int, float and char, it can return a pointer.
- ❖ To make a function return a pointer it has to be explicitly mentioned in the calling function as well as in the function declaration.
- ❖ Three things should be done to avail the feature of functions return pointer.
 1. Declaration of function returning pointer
 2. Declaring pointer and assigning function call
 3. Defining function returning pointer

- ❖ Syntax for declaration of function returning pointer

```
return_type *function_name (arguments);
```

This declaration helps the compiler to recognize that this function returns address.

- ❖ Now declare pointer variable and place the function call

```
ptr = function_name (arguments);
```

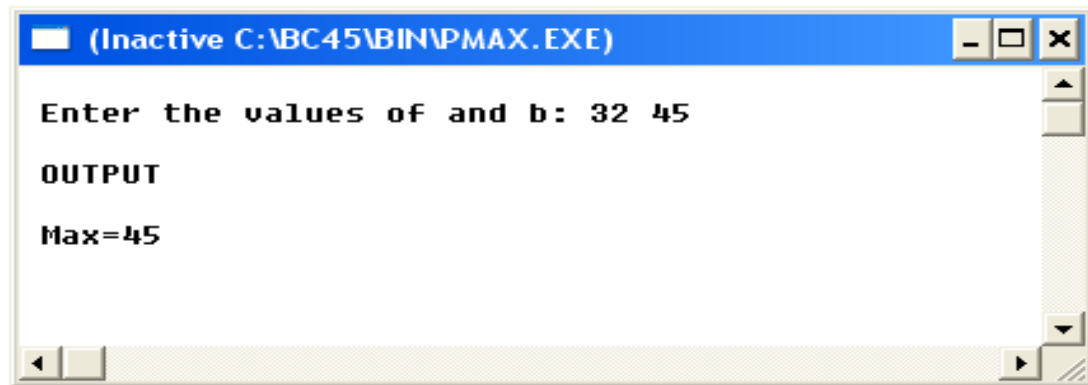
- ❖ After executing above statement ptr consisting of the address that is returned by the function. Remember the return type of the function and pointer type should match here.
- ❖ The function Definition returning pointer takes of the form,

```
return_type *function_name (arguments)
{
    // local declarations
    // executable statements

    return (&variable); Here don't forget to send address
                        with return statement.
}
```

Example:

```
#include<stdio.h>
int *max(int* ,int* );
int main()
{
    int a,b;
    int *ptr;
    printf("\n Enter the values of a and b: ");
    scanf("%d%d",&a,&b);
    ptr=max(&a,&b);
    printf("\n OUTPUT \n");
    printf("\n Max=%d",*ptr);
    getch();
}
int *max(int*pa,int*pb)
{
    if(*pa>*pb)
        return pa;
    else
        return pb;
}
```



The execution of the program as follows,

- ❖ Execution of the program starts at main.
- ❖ Two variables a and b are created and initialized at run-time.
- ❖ A pointer variable is created and initialized with the return value of the function max ().
- ❖ Once the control is transferred from function main () to max (), it got executed and returns the pointer value to main().
- ❖ Here we are having the address of the maximum variable address to display it just use indirection operator (*).

Note: function return pointer does not have any advantage except in the handling of strings.

4.6.2 POINTERS TO FUNCTIONS

Pointer to a function (also known as function pointer) is a very powerful feature of C. Function pointer provides efficient and elegant programming technique. Function pointers are less error prone than normal pointers since we will never allocate or de-allocate memory for the functions.

Every variable with the exception of register has an address. We have seen how we can refer variables of type char, int and float. Through their addresses, by using pointers.

Functions exist in memory just like variables. C will allow you to define pointers to functions. Just like variables, a function name gives the starting address of function stored in memory.

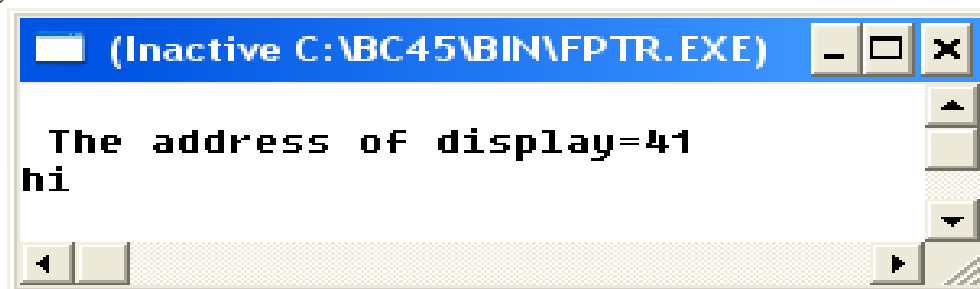
The below code illustrate how to get the address of a function.

```

#include<stdio.h>

int main()
{
    void display();
    printf("\n The address of display=%u",display);
    getch();
}
void display()
{
    printf("hi");
}

```



4.6.2.1 DEFINING POINTERS TO FUNCTIONS

Like declaring pointer variables, we can define pointers to function variables and store the address. The below figure illustrate how function pointer can be represented.

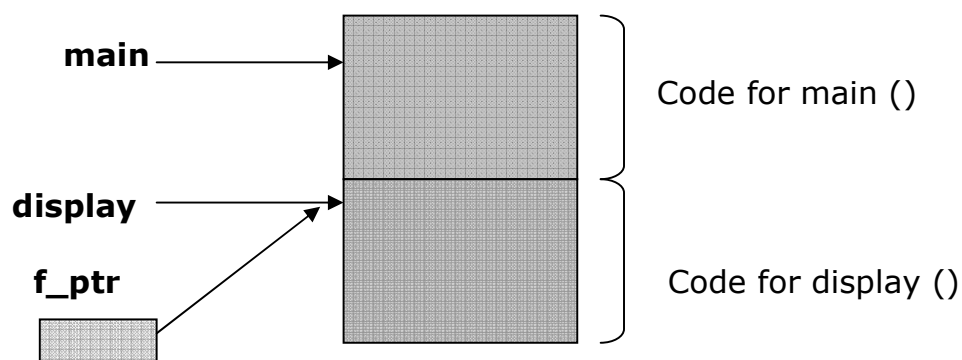


Figure: 4.3. Functions in Memory.

The syntax for declaring pointer to function as follows,

```
return_type (*f_ptr) (arguments);
```

Everything is same as function declaration except the braces for the name, to tell the compiler that this is a function pointer braces are required here and as usual for pointer declarations * is used.

Note that the return type of function pointer, number of arguments and type of arguments must match with the normal function.

The next after the declaration is calling the function using function pointer. before calling takes place we must initialize the function pointer with the address of the function.

The syntax for this assignment,

```
f_ptr=function_name;
```

After this assignment we need to call the function, the syntax associated with the function call is as follows,

```
(*f_ptr)(argument's);
```

This is another way of calling the function. There are no changes in the declaration of the function body.

The below program simulates a simple calculator using function pointers.

```

#include<stdio.h>

int add(int* ,int* );
int sub(int* ,int* );
int mul(int* ,int* );
float div(int* ,int* );
int (*fadd)(int* ,int* );
int (*fsub)(int* ,int* );
int (*fmul)(int* ,int* );
float (*fdiv)(int* ,int* );

int main()
{
    int x,y,ch;
    char c;
    fadd=add;
    fsub=sub;
    fmul=mul;
    fdiv=div;
    printf("\nEnter the values of two operands :");
    scanf("%d%d",&x,&y);
    printf("\n");
    do
    {
        printf("          1.ADDITION          \n");
        printf("          2.SUBTRACTION         \n");
        printf("          3.MULTIPLICATION      \n");
        printf("          4.DIVISION            \n");
        printf("\nEnter your choice :");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:

```

```

    printf("The addition of x=%d and y=%d is = %d",x,y,(*fadd)(&x,&y));
    break;
case 2:
    printf("The subtraction of x=%d and y=%d is = %d",x,y,(*fsub)(&x,&y));
    break;
case 3:
    printf("The multiplication of x=%d and y=%d is = %d",x,y,(*fmul)(&x,&y));
    break;
case 4:
    printf("The division of x=%d and y=%d is = %f",x,y,(*fdiv)(&x,&y));
    break;
}
printf("\nDo you want continue(y/n) ");
fflush(stdin);
scanf("%c",&c);
    }while(c=='y' || c=='Y');
    getch();
}

int add(int *a,int *b)
{
    return(*a+*b);
}

int sub(int *a,int *b)
{
    return(*a-*b);
}
int mul(int *a,int *b)
{
    return(*a * (*b));
}

float div(int *a,int *b)
{
    return(*a/(*b));
}

```

```

(Inactive C:\BC45\BIN\CALPTR.EXE)

Enter the values of two operands :20 10

    1.ADDITION
    2.SUBTRACTION
    3.MULTIPLICATION
    4.DIVISION

Enter your choice :2
The subtraction of x=20 and y=10 is = 10
Do you want continue(y/n) y
    1.ADDITION
    2.SUBTRACTION
    3.MULTIPLICATION
    4.DIVISION

Enter your choice :3
The multiplication of x=20 and y=10 is = 200
Do you want continue(y/n) n
  
```

4.7 POINTERS AND ARRAYS

- ❖ An array is a collection of similar elements stored in contiguous memory locations.
- ❖ When an array is declared, the compiler allocates a base address and sufficient amount of memory depending on the size and data type of the array.
- ❖ The base address is the location of the first element of the array.
- ❖ The compiler defines the array name as a constant pointer to the first element.

4.7.1 POINTERS AND ONE DIMENSIONAL ARRAY

Let us take the following declaration,

```
int num [5] = {1, 2, 3, 4, 5};
```

- ❖ After having this declaration, the compiler creates an array with name **num**, the elements are stored in contiguous memory locations, and each element occupies two bytes, since it is an integer array.

- ❖ The name of the array **num** gets the base address. Thus by writing `*num` we would be able to refer to the zeroth element of the array, that is 1.
- ❖ Then `*num` and `*(num+0)` both refer to the element 1. and `*(num+2)` will refer 3.
- ❖ When we have `num[i]`, the compiler internally converts it to `*(num+i)`.
- ❖ In this light the following notations are same.

$$\text{num}[i] \rightarrow *(num+i) \rightarrow *(i+num) \rightarrow i[\text{num}]$$

- ❖ Then we can also define a pointer and initialize it to the address of the first element of the array (base address).
- ❖ Example, for the above array we can have the following statement,

```
int *ptr=a; (or) int *ptr=&a[0];
```

- ❖ To refer the array elements by using pointer the following notations are used.

$$*ptr \rightarrow *(ptr+i) \rightarrow *(i+ptr) \rightarrow i[ptr]$$

- ❖ `p++` will point to the next location in the array.
- ❖ Accessing array elements by using pointers is always faster than accessing them by subscripts.
- ❖ The below figure shows the array element storage in the memory.

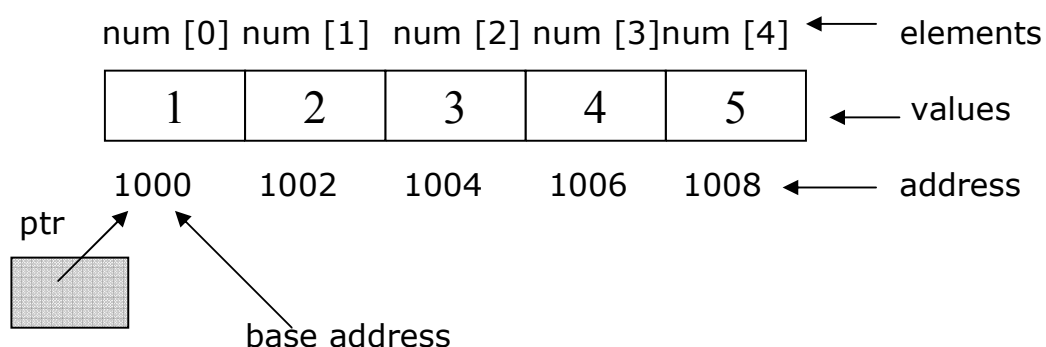
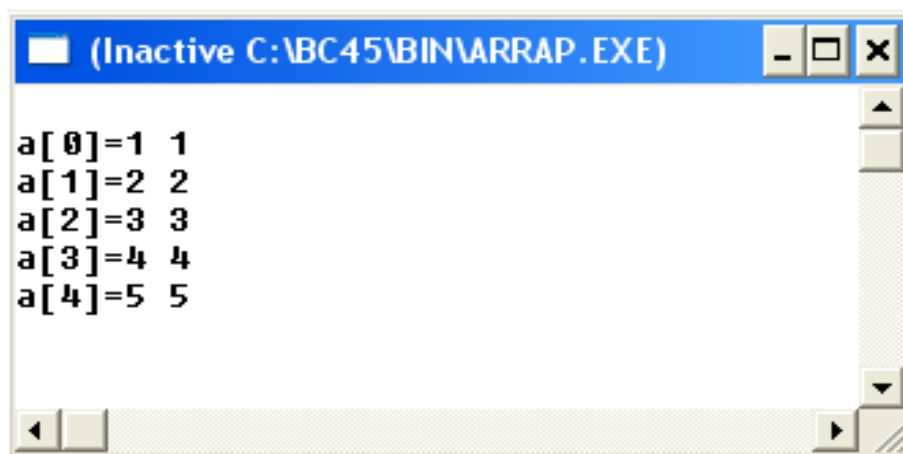


Figure 4.4 Storage representation of array

Example

```
#include<stdio.h>
int main()
{
    int num[5]={1,2,3,4,5};
    int i,*ptr;
    ptr=num;
    for(i=0;i<5;i++)
    {
        printf("\na[%d]=%d %d",i,*ptr,* (num+i));
        ptr++;
    }
    return 0;
}
```



The above program illustrates displaying the array elements using pointers.

Note: Note that the array name `num` is a constant pointer points to the base address, then the increment of its value is illegal, `num++` is invalid.

4.7.2 POINTERS AND TWO DIMENSIONAL ARRAYS

A two dimensional array is an array of one dimensional arrays. The important thing to notice about two-dimensional array is that, just as in a one-dimensional array, the name of the array is a pointer constant the first element of the array, however in 2-D array, the first element is another array.

Let us consider we have a two-dimensional array of integers. When we dereference the array name, we don't get one integer, we get an array on integers. In other words the dereference of the array name of a two-dimensional array is a pointer to a one-dimensional array. Here we require two indirections to refer the elements

Let us take the declaration

`int a [3][4];`

Then following notations are used to refer the two-dimensional array elements,

`a` -----> **points to the first row**
`a+i` -----> **points to i^{th} row**
`*(a+i)` -----> **points to first element in the i^{th} row**
`*(a+i) +j` -----> **points to j^{th} element in the i^{th} row**
`*(*(a+i)+j)`----->**value stored in the i^{th} row and j^{th} column**

		Columns			
		0	1	2	3
Rows	0	1	2	3	4
	1	5	6	7	8
	2	9	10	11	12

Example

```

#include<stdio.h>
int main()
{
    int a[3][4]={
                {1,2,3,4},
                {5,6,7,8},
                {9,10,11,12}
    };

    int i,j;
    for(i=0;i<3;i++)
    {
        for(j=0;j<4;j++)
        {
            printf("\nAddress of a[%d][%d]=%u value=%d",i,j,*(a+i)+j,*(*(a+i)+j));
        }
    }
    printf("\n");
    printf("\n a=%u",a);
    printf("\n a+2=%u",a+2);
    printf("\n *(a+2)=%u",*(a+2));
    printf("\n *a[2]=%d",*a[2]);
    printf("\n *(a+2)+2=%u",*(a+2)+2);
    printf("\n *(*a+2)+2=%d",*(*(a+2)+2));
    getch();
}

```

```

(Inactive C:\BC45\BIN\TWODARA.EXE)

Address of a[0][0]=8326 value=9327
Address of a[0][1]=8328 value=9327
Address of a[0][2]=8330 value=9327
Address of a[0][3]=8332 value=9327
Address of a[1][0]=8334 value=9327
Address of a[1][1]=8336 value=9327
Address of a[1][2]=8338 value=9327
Address of a[1][3]=8340 value=9327
Address of a[2][0]=8342 value=9327
Address of a[2][1]=8344 value=9327
Address of a[2][2]=8346 value=9327
Address of a[2][3]=8348 value=9327

a=8326
a+2=8342
*(a+2)=8342
**a+2=3
*a[2]=9
*(a+2)+2=8346
*(*(a+2)+2)=11

```

4.7.3 POINTERS AND THREE DIMENSIONAL ARRAYS

Three-dimensional array can be thought of array of two-dimensional array. To refer the elements here we require tree indirections.

The notations are,

***(*(a+i) +j) +k** --> points to the address of kth dimension in ith row jth column

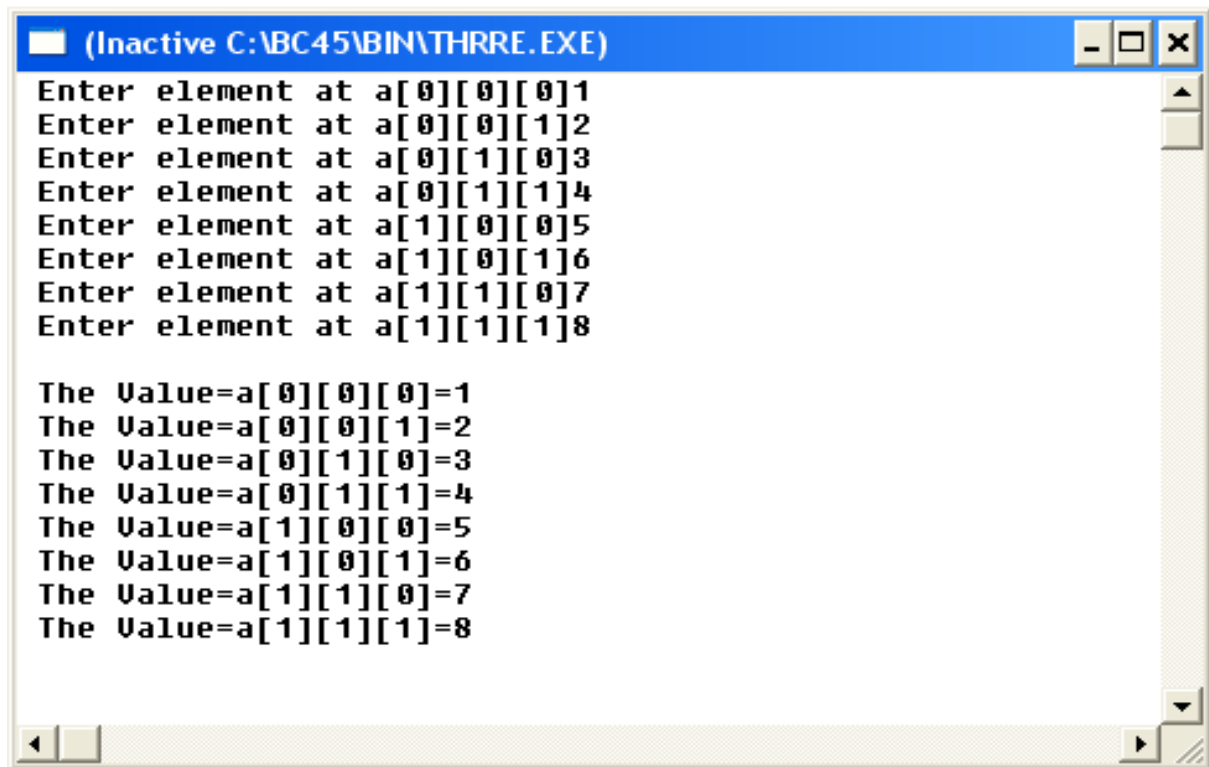
***(*(a+i) +j) +k** --> value stored at kth dimension ith row jth column

Example

```

#include<stdio.h>
int main()
{
    int a[2][2][2],i,j,k;
    for(i=0;i<2;i++)
    {
        for(j=0;j<2;j++)
        {
            for(k=0;k<2;k++)
            {
                printf(" Enter element at a[%d][%d][%d]",i,j,k);
                scanf("%d",&(*(a+i)+j)+k);
            }
        }
    }
    for(i=0;i<2;i++)
    {
        for(j=0;j<2;j++)
        {
            for(k=0;k<2;k++)
            {
                printf("\n The Value=a[%d][%d][%d]=%d",i,j,k,*(a+i)+j+k);
            }
        }
    }
    getch();
}

```



```
(Inactive C:\BC45\BIN\THRRE.EXE)
Enter element at a[0][0][0]1
Enter element at a[0][0][1]2
Enter element at a[0][1][0]3
Enter element at a[0][1][1]4
Enter element at a[1][0][0]5
Enter element at a[1][0][1]6
Enter element at a[1][1][0]7
Enter element at a[1][1][1]8

The Value=a[0][0][0]=1
The Value=a[0][0][1]=2
The Value=a[0][1][0]=3
The Value=a[0][1][1]=4
The Value=a[1][0][0]=5
The Value=a[1][0][1]=6
The Value=a[1][1][0]=7
The Value=a[1][1][1]=8
```

4.8 FUNCTIONS AND ARRAYS

To process arrays in large program, we have to be able to pass them to function. We can pass arrays in two ways,

- ❖ Passing individual elements
- ❖ Passing entire array elements

Passing individual elements

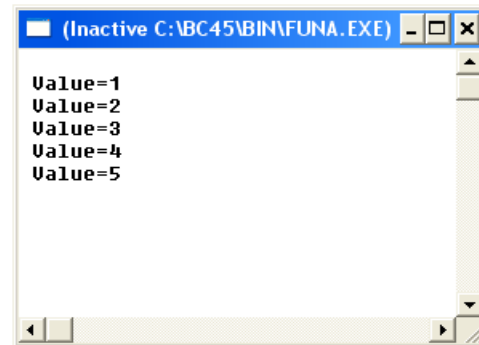
We can pass individual elements by either their data values or by passing their addresses.

1. Passing Data Values

- ❖ This is same as passing data values. Here we are passing an individual array element at a time to the function.
- ❖ The called function cannot tell whether the value it receives comes from an array, or a variable.

Example

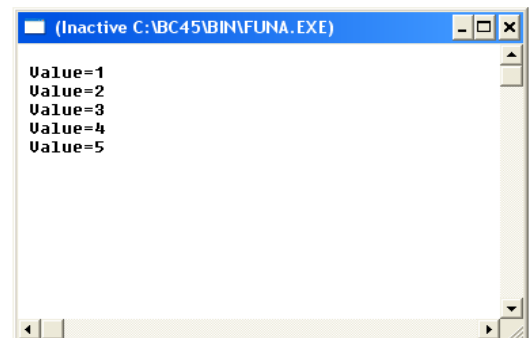
```
#include<stdio.h>
void fun(int );
int main()
{
    int a[5]={1,2,3,4,5};
    int i;
    for(i=0;i<5;i++)
    {
        fun(a[i]);
    }
    return 0;
}
void fun(int x)
{
    printf("\n Value=%d",x);
}
```

**2. Passing Addresses of array elements**

Here we are passing the individual elements address. the called function requires declaration of arguments as pointer variables.

Example

```
#include<stdio.h>
void fun(int* );
int main()
{
    int a[5]={1,2,3,4,5};
    int i;
    for(i=0;i<5;i++)
    {
        fun(&a[i]);
    }
    return 0;
}
void fun(int *px)
{
    printf("\n Value=%d",*px);
}
```

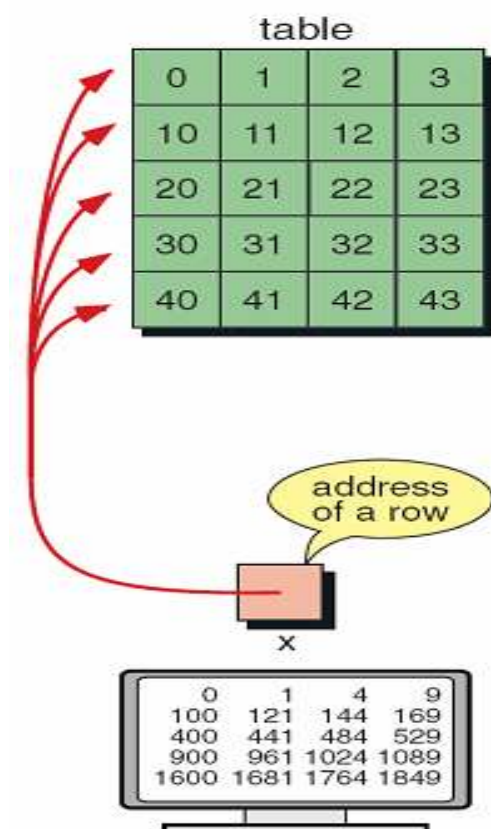


Passing the Entire Array

Here we see the first situation in which C does not pass values to a function. The reason for this change is that a lot of memory and time would be used in passing large arrays every time we wanted to use one in function.

For example, if an array containing 1200 elements were passed by value to a function, another 1200 elements would have to be allocated in the called function and each element would be copied from one array to the other. So, instead of passing the whole array, C passes the address of the array.

In C, the name of the array value is address of the first element in the array. When we pass the name, it allows the called function to refer to the array back in the calling function. Passing both two dimensional array and one dimensional are same but subscripts are changed.



```
#include<stdio.h>
void square (int [] [4]);
int main ()
{
    int a[5][4]={
        {0, 1, 2, 3},
        {10, 11, 12, 13},
        {20, 21, 22, 23},
        {30, 31, 32, 33},
        {40, 41, 42, 43}
    };

    square(a);
    return 0;
}
void square(int x[5][4])
{
    int i,j;
    for(i=0;i<5;i++)
    {
        for(j=0;j<4;j++)
        {
            printf ("\t%d",x[i][j]*x[i][j]);
        }
        printf("\n");
    }
}
```


The above program find the squares of the elements of the array .Here we passed the name of the array as an argument to the function an in the called function the formal argument is created and points to the elements of the calling function argument array.

4.9 INTER FUNCTION COMMUNICATION

The communication between the calling function and called function for the exchange of data is referred as inter function communication.

The dataflow between the calling function and called function can be divided into three strategies:

1. **Downward flow**
2. **Upward flow**
3. **Bi-directional flow**

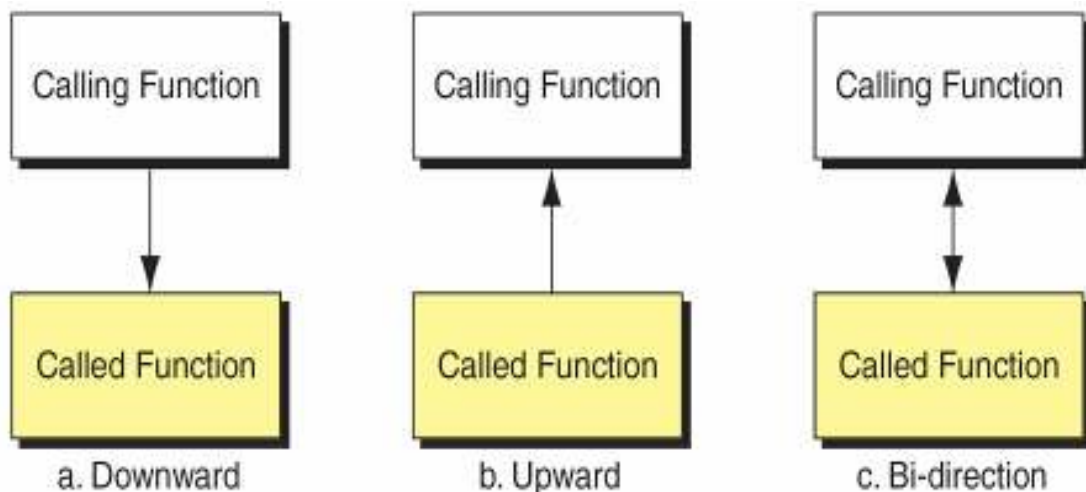


Figure 4.5 Data flow strategies

4.9.1 DOWNWARD FLOW

- ❖ In downward communication, the calling function sends data to the called function. NO data flows in opposite direction. Downward communication is one-way communication.
- ❖ In this strategy, copies of the data items are passed from the calling function to the called function.
- ❖ The called function may change the values passed, but the original values in the calling function remain unchanged.
- ❖ The call by value mechanism in C is a perfect solution for the communication in the downward direction.

- ❖ The figure 4.6 illustrates the downward communication. Two data items are passed from main to the downFun function. One data value is a literal (value); the other is the value of a variable.

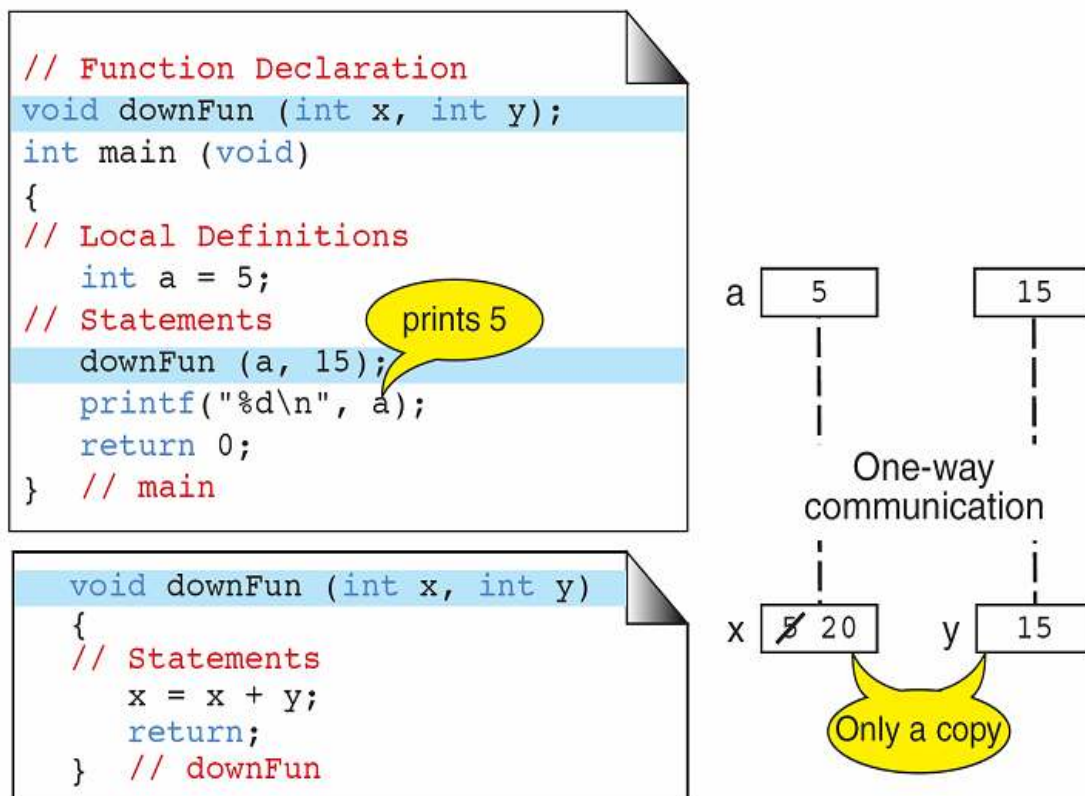


Figure 4.6 Downward communication

Rules for Downward communication

- ❖ Use values in the function call to pass data.
- ❖ Use appropriate data values in the function parameter list to receive the data values.
- ❖ Use the parameter identifiers in the called function to access the local copies of the data.

4.9.2 UPWARD COMMUNICATION

Upward communication occurs when the called function sends data back to the calling function without receiving any data from it.

- ❖ C provides only one upward direction flow, the return statement. While it works well, only one data item can be returned.
- ❖ The only way that a called function can pass multiple data an item up to the calling function is to access variables in the calling function and deposit the data there.
- ❖ However, C does not allow us to directly reference a variable in the calling function. In other words, we cannot access a variable in the calling function by its identifier.
- ❖ The solution is for the calling function to pass address of the variable to the called function.
- ❖ To get the address of a variable, we use the address operator (**&**). The following example passes the address of **a** and **b** to function upFun ().

upFun (&a, &b);

- ❖ The called function needs to declare that the parameter is to receive an address in other words, it needs to create an address variable.
- ❖ To declare an address variable, we can use an asterisk (*****). For Example, to define the function upFun (), we can use the following header.

upFun (int *ax, int *ay);

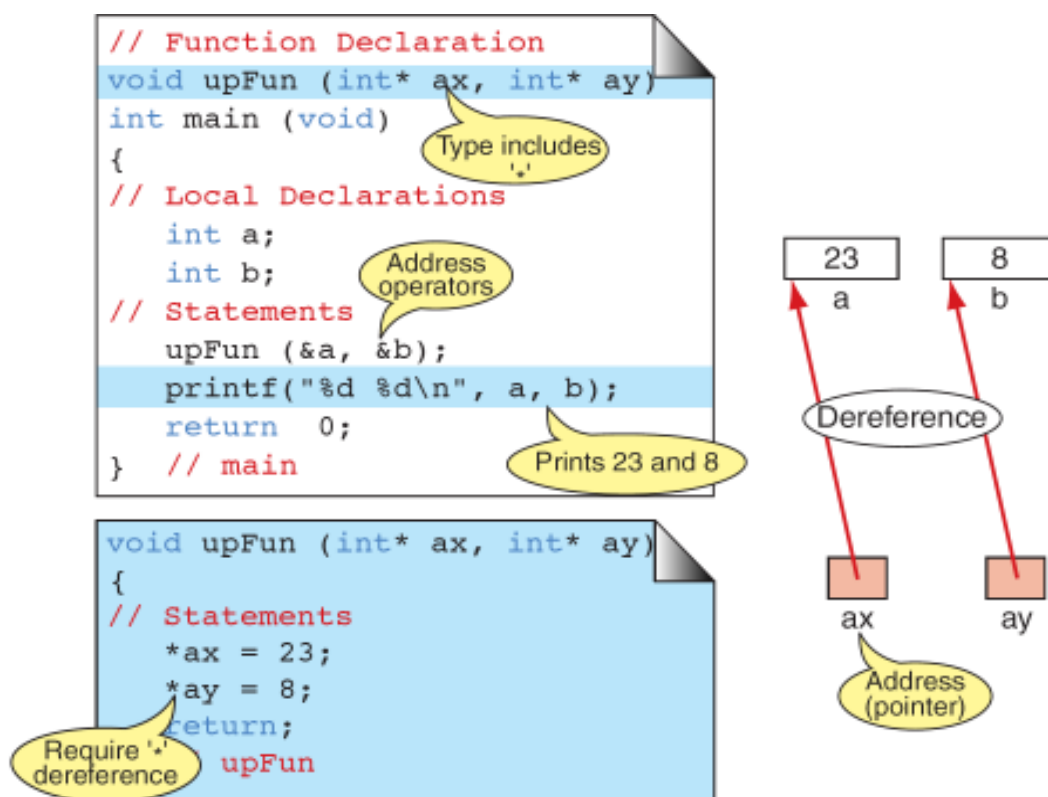


Figure 4.7 Upward communication

Rules for Upward Communication

To send data from the called function to the calling function:

1. We need to use the & symbol in front of the data variable when we call the function.
2. We need to use the * symbol after the data type when we declare the address variable
3. We need to use the * in front of the variable when we store data indirectly.

4.9.3 BI-DIRECTIONAL FLOW

- ❖ Bi- directional communication occurs when the calling function sends data down to the called function.
- ❖ Here the called function sends data up to the calling function.
- ❖ Here indirection must be used in both sides of the assignment statement.
- ❖ The rules for the Bi-directional compunction are same as upward communication.

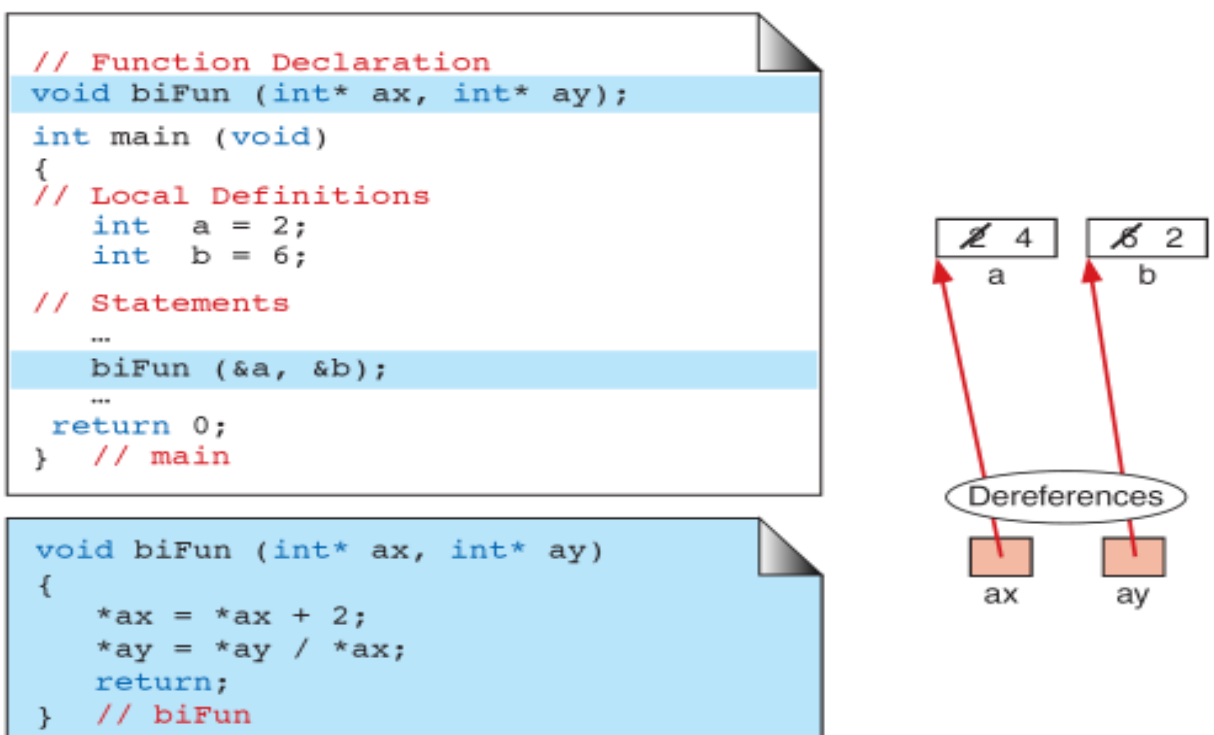


Figure 4.8 Bi-direction communication

4.10 TYPE QUALIFIERS IN C

The type qualifier associated with some values which do not change frequently. There are three type qualifiers in C,

1. **const**
2. **volatile**
3. **restrict**

When a storage class, such as static, and a type qualifier are both needed, the storage class comes before the type qualifier.

4.10.1 CONSTANTS

- ❖ The keyword for the constant type qualifier is **const**.
- ❖ A constant object must be initialized when it is declared because it cannot be changed later.
- ❖ The syntax for declaring constant is

const type var_name = value;

- ❖ Example,

const float pi=3.14;

- ❖ Remember that, we cannot change the value of constant once it is initialized.
- ❖ A string constant can be declared as follows,

const char str[] ="hello";

Pointers and Constants

Pointers can also be defined as constants. Depending on how they are coded, however, three different variations can occur.

1. The pointer itself is a constant.
2. The object being pointed to is constant.
3. Both the pointer and its object are constants.

1. The pointer itself is a constant

- ❖ When the keyword **const** is associated with the identifier, that it is placed after the data type and before the identifier, the pointer itself is a constant.
- ❖ This declaration tells that its contents cannot be changed after initialization.
- ❖ Can change what it points to but cannot change the value of the object it points to.
- ❖ This form turns out to be useful for passing objects to functions when using "pass-by-reference" semantics.
- ❖ Example,

```
int i = 5, j = 6;
const int *p = &i;
p = &j;      /* Valid. p now points to j. */
*p = i;      /* Invalid. Cannot change j via p. */
```

2. The object being pointed to is constant

- ❖ When the keyword **const** is associated with the type, that is, it is placed before the type, then the object being referenced is a constant, but the pointer itself is not.
- ❖ Can change value of pointed-to object, but pointer must always refer to the same address.
- ❖ Example,

```
int i = 5, j = 6;
int * const p = &i;
*p = j;      /* Valid. i is now 6 */
p = &j;      /* Invalid. p must always point to i. */
```

3. Both the pointer and its object are constants

- ❖ To indicate that both the pointer and the object it points to are constant, use the keyword **const** twice.
 - ❖ Example,
- ```
int i = 5, j = 6;
const int * const p = &i;
p = j; / Invalid. i cannot be changed via p. */
p = &j; /* Invalid. p must always point to i. */
```

### 4.10.2 VOLATILE

The **volatile** qualifier tells the computer that an object may be changed by entities other than this program. When the compiler owns an object, it can store and handle it in any way necessary to optimize the program. As an example, a C compiler may think that it is more efficient to remove an object from RAM memory and put it into a register.

- ❖ However, sometimes objects must be shared between your program and some other facilities outside the C program.
- ❖ To tell the compiler that the object is shared, we declare it as type **volatile**.
- ❖ The following shows how an integer or a pointer can be declared volatile.

```
volatile int x;
volatile int* ptr;
```

### 4.10.3 RESTRICTED

The **restrict** qualifier, which is used only with pointers, indicates that the pointer is only the initial way to access the dereferenced data. It provides more help to the compiler for optimization.

Let us at the following code:

```
int *ptr;
int i=0;
ptr=&i;
...
...
*ptr = *ptr+4;
...
*ptr = *ptr+5;
...
```

Here, the compiler cannot replace two statements `*ptr=*ptr+4` and `*ptr=*ptr+5` by one statement `*ptr=*ptr+9` because it does not know if the variable **i** can be accessed directly or through other pointers.

Now, let us look a same code using the restrict qualifier:

```
restrict int *ptr;
int i=0;
ptr=&i;
...
...
*ptr = *ptr+4;
...
...
*ptr = *ptr+5;
...
```

Here, the compiler replaces the two statements by one statement `*ptr=*ptr+9`, because it is sure that variable `i` cannot be accessed through other resources.

#### 4.11 STRINGS IN C

- ❖ A string is a sequence/array of characters.
- ❖ C has no native string type; instead we use arrays of char.
- ❖ A special character, called a **"null"**, is important because it is the only way the functions that work with a string can know where the string ends.
- ❖ This may be written as **'\0'** (zero not capital 'o'). This is the only character whose ASCII value is zero.
- ❖ Depending on how arrays of characters are built, we may need to add the null by hand, or the compiler may add it for us.

The following operations performed on character strings,

- ❖ Reading and Writing strings.
- ❖ Combining Strings together.
- ❖ Copying one string to another.
- ❖ Comparing strings for equality.

##### 4.11.1 DECLARING AND INITIALIZING STRING VARIABLES

###### Declaring a String

A string variable is a valid C variable name and always declared as an array.



The general form of declaration of a string variable is,

```
char string_name [size];
```

- ❖ The size determines the number of characters in the string\_name.
- ❖ When the compiler assigns a character string to a character array, it automatically supplies a null character('\0') at the end of the string.
- ❖ The size should be equal to the maximum number of characters in the string plus one.

### Initializing String Variables

Character arrays may be initialized when they are declared. C permits a character array to be initialized in one of the following forms,

- ❖ Initializing locations character by character.
- ❖ Partial array initialization.
- ❖ Initializing without specifying the size.
- ❖ Array initialization with a string constant.

### Initializing locations character by character

If you know all the characters at compile time, you can specify all your data within brackets:

Example,

```
char s[6]={ 'h','e','l','l','o' };
```

The compiler allocates 6 memory locations ranging from 0 to 5 and these locations are initialized with the characters in the order specified. The remaining locations are automatically initialized to null characters as shown in the below figure 4.9.

| s[0] | s[1] | s[2] | s[3] | s[4] | s[5] |
|------|------|------|------|------|------|
| h    | e    | l    | l    | o    | \0   |
| 1000 | 1001 | 1002 | 1003 | 1004 | 1005 |

→ Address

Figure 4.9: Initializing Location Character by character

**Note:** It is the programmer responsibility to allocate sufficient memory so as to accommodate NULL character at the end. Note that The ASCII values of characters are stored in the memory.

### Partial Array Initialization

If the number of characters values to be initialized is less than the size of the array, then the characters are initialized in the order from 0<sup>th</sup> location. The remaining locations will be initialized to NULL automatically. Consider the following initialization,

```
char s[10]={‘h’,‘e’,‘l’,‘l’,‘o’};
```

The above statement allocates 10 bytes for the variable **s** ranging from 0 to 9 and initializes first 5 locations with the characters. The remaining locations are automatically filled with NULL as shown in below figure 4.10.

|      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|
| s[0] | s[1] | s[2] | s[3] | s[4] | s[5] | s[6] | s[7] | s[8] | s[9] |
| h    | e    | l    | l    | o    | \0   | \0   | \0   | \0   | \0   |
| 200  | 201  | 202  | 203  | 204  | 205  | 206  | 207  | 208  | 209  |

Figure 4.10 Partial Array Initialization

### Initialization Without Size

If we omit the size of the array, but specify an initial set of characters, the compiler will automatically determine the size of the array. This way is referred as initialization without size.

```
char s[]={‘h’,‘e’,‘l’,‘l’,‘o’};
```

In this declaration, even though we have not specified exact number of characters to be used in array **s**, the array size will be set of the total number of initial characters specified and appends the NULL character.. Here, the compiler creates an array of 6 characters. The array **s** is initialized as shown in Figure 4.11.

|      |      |      |      |      |      |
|------|------|------|------|------|------|
| s[0] | s[1] | s[2] | s[3] | s[4] | s[5] |
| h    | e    | l    | l    | o    | \0   |
| 1000 | 1001 | 1002 | 1003 | 1004 | 1005 |

Figure 4.11: Initializing With out size

### Array Initialization with a String Constant

It takes of the following form,

```
char s[]="hello";
```

Here the length of the string is 5 bytes, but size is 6 bytes. The compiler reserves **5+1** memory locations and these locations are initialized with the characters in the order specified. The string is terminated by Null as shown in the figure 4.12.

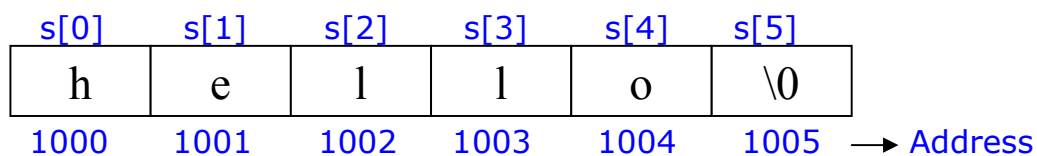


Figure 4.12: Array Initializing With a String

Here are some illegal statements representing initialization of strings,

- ❖ The following declaration creates character array only not a string

```
char s[5]={ 'h','e','l','l','o' }; //no location for appending NULL
```

- ❖ The following declaration is illegal.

```
char str[3]="Good"; //size is less than the total characters
```

- ❖ We cannot separate the initialization from declaration.

```
char str3[5];
str3="Good"; Is not allowed.
```

- ❖ Similarly,

```
char s1[4]="abc";
char s2[4];
s2=s1; /* Error */
```

**Note:** Observe the difference between the following ,

- 0 --> it is an integer zero. Occupies two bytes of memory.
- '0' --> it is a character constant .It occupies one byte.

- '0" --> it is a string constant. It occupies two bytes. The first byte contains the value 0 and second byte contains \0.
- '\0' --> it is Null character and occupies 1 byte.
- "\0" --> it is a string containing a null-character. It occupies 2 bytes. Together, the string "\0" occupies two bytes.

#### 4.11.2 STRING INPUT/OUTPUT FUNCTIONS

Strings can be read from the keyboard and can be displayed onto the monitor using the following I/O functions.

##### **Formatted Input Function-scanf ()**

The string can be read using the scanf function also. The format specifier associated with the string is **%s**.

Syntax for reading string using scanf function is

**scanf ("%s", string\_name);**

##### **Disadvantages**

- ❖ The termination of reading of data through scanf function occurs, after finding first white space through keyboard. White space may be new line (\n), blank character, tab(\t).
- ❖ For example if the input string through keyboard is "hello world" then only "hello" is stored in the specified string.

##### **Formatted Output function-printf ()**

The various options associated with printf ():

1. Field width specification
2. Precision specifier
3. Left Justification

##### **1. Field Width Specification**

Syntax: **%ws**

- ❖ W is the field with specified width.
- ❖ S indicates that the string is being used.

### **NOTE:**

1. If the string to be printed is larger than the field width w, the entire string will be printed.
2. If the string to be printed is smaller than the field width w, then appropriate numbers of blank spaces are padded at the beginning of the string so as to ensure that field width w is reached.

#### **Example:**

```
#include<stdio.h>
void main ()
{
 char s[]="RAMANANDA";
 printf ("%4s\n", s);
 printf ("%15s",s);
}
```

#### **OUTPUT:**

```
RAMANANDA
 RAMANANDA
```

## **2. Precision Specifier**

Syntax:     **%w.ns**

- ❖ W is the field specified width
- ❖ N indicates that first n characters have to be displayed. This gives precision.
- ❖ S indicates that the string is being used.

#### **Example:**

```
#include<stdio.h>
#include<conio.h>
void main()
{
 char s []={'R','A','M','A','N','A','N','D','A'}:
 clrscr();
 printf("%0.2s\n",s);
 printf("%9.4s\n",s);
 printf("%9.3s\n",s);
 printf("%3.5s",s);
 getch();
}
```

#### **OUTPUT:**

```
RA
 RAMA
 RAM
 RAMAN
```

**NOTE:**

- ❖ The string is printed right justification by default.
- ❖ If  $w > n$ ,  $w$  columns are used to print first  $n$  characters .example 2nd and 3rd printf statements.
- ❖ If  $w < n$ , minimum  $n$  columns are used to print first  $n$  characters. Example, 1st and 4th printf statements.

**3. Left justification****Syntax:**                      **%-w.ns**

- ❖ - just before  $w$  indicates that string is printed using left justification.
- ❖  $W$  is the field with specified width.
- ❖  $S$  indicates that the string is being printed.

**Example:**

```
#include<stdio.h>
#include<conio.h>
void main ()
{
char s []={'R','A','M','A','N','A','N','D','A'}:
clrscr();
printf ("%%-0.2s\n", s);
printf ("%%-9.4s\n",s);
printf ("%%-9.3s\n",s);
printf ("%%-3.5s",s);
getch();
}
```

**OUTPUT:**

```
RA
RAMA
RAM
RAMAN
```

**Character I/O from Keyboard**

To read characters from the keyboard and write to screen it takes the following form:

```
c = getchar(); //reads one character from the keyboard
putchar(c); // display the character on the monitor
```

### Un-Formatted Input Function-gets ()

C provides easy approach to read a string of characters using gets() function.

Syntax:     **gets (string\_name);**

The function accepts string from the keyboard. The string entered includes the white spaces. The input will terminate only after pressing <Enter Key>. Once the <Enter key > is pressed ,a null character(\0) appended at the end of the string.

### Advantage

- ❖ It is capable of reading multiple words from the keyword.

### Un-Formatted Output Function- puts ()

- ❖ It is a library function available in <stdio.h>.
- ❖ This a one parameter function and is invoked as under:

**puts(str);**

- ❖ Where str is a string variable containing a string value.

### 4.11.3 TWO DIMENSIONAL ARRAY OF CHARACTERS

It is also referred as table of strings. It can be initialized as follows:

**type variable-name[][];**

The first subscript gives the number of names in the array.  
The second subscript gives the length of each item in the array.

```
Example:char list[6][10]={
 "akshay",
 "parag",
 "raman",
 "srinivas",
 "gopal",
 "rajesh"
};
```

The names would be store in the memory as shown below.

|       |   |   |   |   |   |    |    |   |    |  |
|-------|---|---|---|---|---|----|----|---|----|--|
| 65454 | a | k | s | h | a | y  | \0 |   |    |  |
| 65464 | p | a | r | a | g | \0 |    |   |    |  |
| 65474 | r | a | m | a | n | \0 |    |   |    |  |
| 65484 | s | r | i | n | i | v  | a  | s | \0 |  |
| 65494 | g | o | p | a | l | \0 |    |   |    |  |
| 65504 | r | a | j | e | s | h  | \0 |   |    |  |

#### 4.11.4 STRING HANDLING FUNCTIONS

The C Library provides a rich set of string handling functions that are placed under the header file <string.h>.

##### strcat () function:

The strcat function joins two strings together. It takes of the following form:

**strcat(string1,string2);**

String1 and string2 are character arrays. When the function strcat is executed, string2 is appended to string1. It does so by removing the null character at the end of string1 and placing string2 from there.

strcat function may also append a string constant to a string variable. The following is valid.

**strcat(part1,"Good");**

C permits nesting of strcat functions. Example:

**strcat(strcat(string1,string2),string3);**



**Example**

```
#include <stdio.h>
#include <string.h>
main ()
{
 char opening[255] = "And, the Oscar goes to... ";
 char winner[255] = "American Beauty";
 strcat (opening, winner);
 printf ("%s", opening);
 getchar();
}
```

OUTPUT:

And, the Oscar goes to... American Beauty

When using strcat, be careful that you do not overrun the array size. For example, do not append a 255 char word to opening.

**strcmp () function:**

The strcmp function compares two strings identified by the arguments and has the value 0 if they are equal. If they are not, it has the numeric difference between the first non matching characters in the strings. It takes the following form:

**strcmp(str1,str2);**

- ❖ return value less than 0 means "str1" is less than "str2"
- ❖ return value 0 means "str1" is equal to "str2"
- ❖ return value greater than 0 means "str1" is greater than "str2"

String1 and string2 may be string variables or string constants.

Example: strcmp(name1,name2);  
 strcmp(name1,"John");  
 strcmp("their" ,"there");

**Example: Password Protection**

```
#include <stdio.h>
#include <string.h>
main ()
{
 char password[255];
 printf ("Enter password: ");
 scanf ("%s", password);
 while (strcmp (password, "bluemoon") != 0) {
 printf ("Access Denied. Enter Password: ");
 scanf ("%s", password); }
 printf ("Welcome!");
 getch();}
```

**strcpy () function:**

it takes the following form:

**strcpy(string1,string2);**

and assign the contents of string2 to string1.

String2 may be a character array variable or a string constant.

Example:           strcpy(city ,"Delhi");  
                      strcpy(city1,city2);

```
#include <stdio.h>
#include <string.h>
```

```
main ()
```

```
{
```

```
 char word1[] = "And, the winner is....";
```

```
 char word2[255];
```

```
 strcpy (word2, word1);
```

```
 printf ("%s", word2);
```

```
 getch ();
```

```
}
```

Output:

And, the winner is....

**strlen () function:**

This function counts and returns the number of characters in a string. It takes the form

**n=strlen(string);**

Where n is an integer variable, which receives the value of the length of the string. The counting ends at the first null character.

**Example:**

```
#include<stdio.h>
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```
 char name[100]="Gore";
```

```
 printf ("%d", strlen (name));
```

```
 getch();
```

```
}
```

**Output:** 4

Note, however, that the size of the array is 100

**strrev () function**

Reverses the contents of the string. It takes of the form

**strrev(string);**

**Example:**

```
#include<stdio.h>
#include<string.h>
void main()
{
 char s[]="hello";
 strrev(s);
 puts(s);
 getch();
}
```

**OUTPUT:**

olleh

**strstr () function:**

It is a two-parameter function that can be used to locate a sub-string in a string. It takes the form:

**strstr (s1, s2);**

**Example: strstr (s1,"ABC");**

The function strstr searches the string s1 to see whether the string s2 is contained in s1.If yes, the function returns the position of the first occurrence of the sub-string. Otherwise, it returns a NULL pointer.

Example: if (strstr (s1, s2) ==NULL)  
           printf ("substring not found");  
       else  
           printf ("s2 is a substring of s1");

We also have the functions to determine the existence of a character in a string.

Example: **strchr (s1,'m');**

Will locate the first occurrence of the character 'm'.

Example: **strrchr(s2,'m');**

Will locate the last occurrence of the character 'm'.

| Functions included in <string.h> |          |                                                |
|----------------------------------|----------|------------------------------------------------|
| Operation                        | Function | Description                                    |
| <b>Copying</b>                   |          |                                                |
|                                  | memcpy   | Copies a block of memory                       |
|                                  | memmove  | Move block of memory                           |
|                                  | strcpy   | Copy string                                    |
|                                  | strncpy  | Copy n number characters from string           |
| <b>Concatenation</b>             |          |                                                |
|                                  | strcat   | Concatenate strings                            |
|                                  | strncat  | Append n number of characters from string      |
| <b>Comparison</b>                |          |                                                |
|                                  | memcmp   | Compare two blocks of memory                   |
|                                  | strcmp   | Compare two strings                            |
|                                  | strcoll  | Compare two strings using locale               |
|                                  | strncmp  | Compare first n characters of two strings      |
|                                  | strxfrm  | Transform string using locale                  |
| <b>Searching</b>                 |          |                                                |
|                                  | memchr   | Locate character in block of memory            |
|                                  | strchr   | Locate first occurrence of character in string |
|                                  | strcspn  | Get span until character in string             |
|                                  | strpbrk  | Locate character in string                     |
|                                  | strrchr  | Locate last occurrence of character in string  |
|                                  | strspn   | Get span of character set in string            |
|                                  | strstr   | Locate substring                               |
|                                  | strtok   | Split string into tokens                       |
| <b>Other</b>                     | strrev   | reverse the content of the string              |
|                                  | memset   | Fill block of memory                           |
|                                  | strerror | Get pointer to error message string            |
|                                  | strlen   | Get string length                              |

Table 4.2: String Functions in C

#### 4.11.3 CHARACTER POINTER

Suppose we wish to store "Hello". We may either store it in a string or we may ask the C compiler to store it at some location in memory and assign the address of the string in a char pointer.

Consider the following declaration with string initialization,

```
char *p="hello";
```

Here the string length is 5 bytes. So the compiler allocates 6 bytes memory locations. Probably the characters are stored in the constant memory area of the computer, and the pointer p points to the base address as shown in the below figure 4.13.

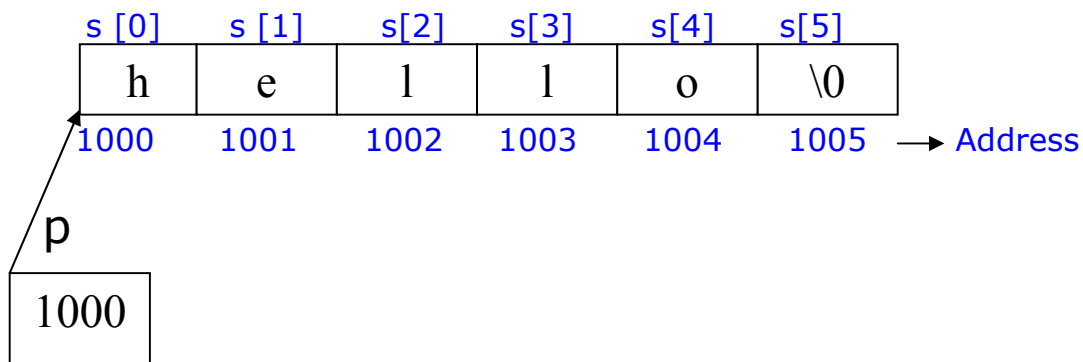


Figure 4.13: Initializing using Character Pointer

- ❖ We cannot assign a string to another. But, we can assign a char pointer to another char pointer.

```
Example: char *p1="hello";
char *p2;
p1=p2; //valid
printf ("%s", p1); //will print hello
```

- ❖ \*p will refer to a particular character only, p will refer whole string.

#### 4.11.4 POINTERS AND STRINGS

A string is a character array. so the name of the string it self is a pointer. Like referring array elements also string characters also refereed with its name.

Example:

```
char s [] ="hello";
```

The following notations are used to refer individual characters

**s[i] --> \*(s+i) --> \*(i+ s)** all will point to the ith character in the given string.

We can also use pointer variable to refer the string elements.

```
char s []="hello";
char *ptr;
ptr=s; // now ptr points to the base address of the string.
```

then the following notations are same,

**\*ptr --> \*(ptr+i) --> \*(i+ptr)** will point value at the ith character.

**Example:** // illustrates displaying characters using pointer

```
#include<stdio.h>
void main ()
{
 char s [] ="hello";
 char *ptr;
 ptr=s;
 while (*ptr! ='\0')
 {
 printf (" %c",*ptr);
 ptr++;
 }
}
```

OUTPUT

h e l l o

## 4.12 ARRAY OF POINTERS

- ❖ A pointer variable always contains an address; an array of pointers would be nothing but a collection of addresses.
- ❖ The addresses present in the array of pointers can be addresses of variables or addresses of array elements or any other addresses.
- ❖ The major application of this is in handling data tables efficiently and table of strings.
- ❖ All rules that apply to an ordinary array apply to the array of pointers as well.
- ❖ The Syntax for declaration of array of pointers as follows,

**data type \*arr\_ptr [size];**

- ❖ This declaration tells the compiler arr\_ptr is an array of addresses, pointing to the values of data type.

- ❖ Then initialization can be done same as array element initialization. Example **arr\_ptr [3] = &var**, will initialize 3<sup>rd</sup> element of the array with the address of var.
- ❖ The dereferencing operator is used as follows

**\*(arr\_ptr [index])** --> will give the value at particular address.

- ❖ Look at the following code **array of pointers to ordinary Variables**

### Example

```
#include<stdio.h>
void main ()
{
 int i=1, j=2, k=3, l=4, x;
 int *arr [4]; //declaration of array of pointers
 arr [0] =&i; //initializing 0th element with address of i
 arr [1] =&j; //initializing 1st element with address of j
 arr [2] =&k; //initializing 2nd element with address of k
 arr [3] =&l; //initializing 3rd element with address of l
 for (x=0; x<4; x++)
 printf ("\n %d",*(arr[x]));
}
```

### OUTPUT

1  
2  
3  
4

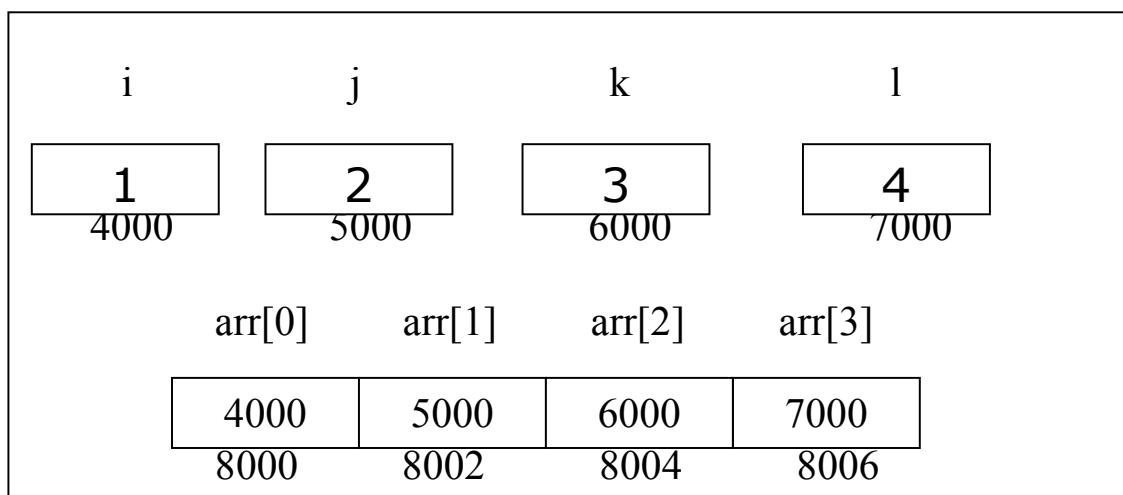


Figure 4.14 Memory organization

The above figure 4.14 shows contents and the arrangement of the array of pointers in memory. Here, arr contains the addresses of int variables i, j, k and l. The for loop is used to print the values present at these addresses.

A two-dimensional array can be represented using pointer to an array. But, a two-dimensional array can be expressed in terms of array of pointers also.

Using array of pointers a two dimensional array can be defined as,

**data type \*arr\_ptr [size];**

where data type refers to the data type of the array. arr\_ptr refers to the name of the array and size is the maximum number of elements in the row.

**Example int \*arr [3];**

Here, p is an array of pointers, and then following notations are used to refer elements.

**p [i] --> points the address of the element ith row,**  
**p[i] +j --> points the address of the element ith row and jth column**  
**\*(p[i] +j) --> value at ith row and jth column.**

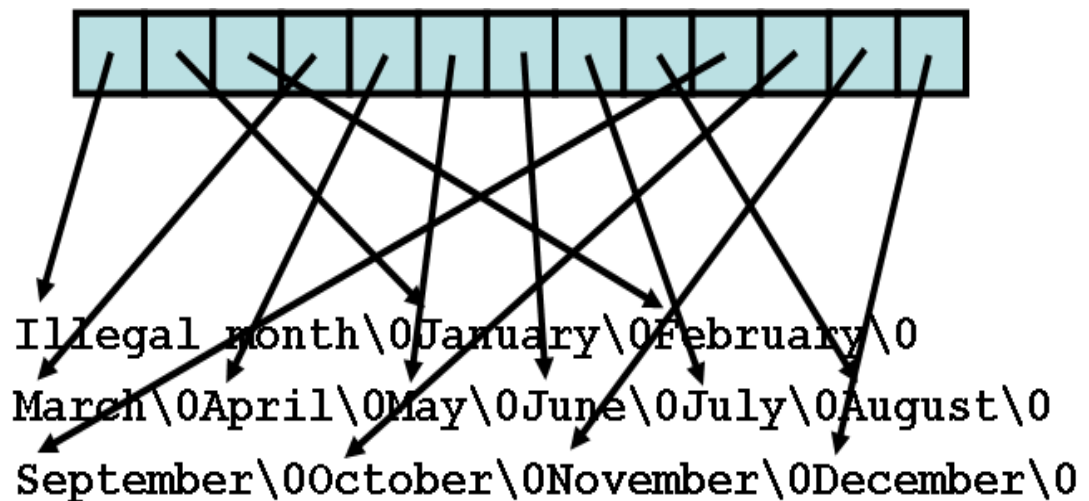
### Array of pointers and Strings

- ❖ Each element of the array is a pointer to a data type (in this case character).
- ❖ A block of memory (probably in the constant area) is initialized for the array elements.
- ❖ Declaration:  
**char \*names [10];** // array of 10 character pointers.
- ❖ In this declaration names [] is an array of pointers. It contains base address of respective names. That is, base address of first name is stored in name [0] etc., a character pointer etc.
- ❖ The main reason to store array of pointers is to make more efficient use of available memory.



```
include <stdio.h>
int main ()
{
 char *name [] = {
 "Illegal month",
 "January", "February", "March",
 "April", "May", "June",
 "July", "August", "September",
 "October", "November", "December"
 };
}
```

The pointers are initialized like so



**Note:** When we are using an array of pointers to strings we can initialize the string at the place where we are declaring the array, but we cannot receive the string from keyword using scanf ().

## Exercise Programs

1. The names of employees of an organization are stored in three arrays, namely, first name, second name, and last name. Write a program to concatenate the three parts into one string to be called name.
2. Write a C program to read in an array of integers. Instead of using subscripting, however, employ an integer pointer that points to the elements currently being read in and which is incremented each time.
3. Write a C program to find number of words, blank spaces, special characters, digits and vowels of a given text using pointers.
4. Write a C program using pointer for string comparison.
5. Write a C program to find out the presence of a substring in a given string using pointers.
6. Write a C program to find the length of a given string using pointers.
7. Write a program to reverse the strings stored in array of pointers.
8. Write a program to find rank of a matrix using pointers.
9. Write a C program using pointers to read in an array of integers and print its elements in reverse order.
10. Write a C program to arrange the given numbers in ascending order using pointers.
11. Write a 'C' program to find factorial of a given number using pointers.
12. Write a 'C' program to arrange the given names in alphabetical order using pointers.
13. Write a 'C' Program to compute the sum of all elements stored in an array using pointers.

14. Write a C program to find the desired element in an array of N elements. Use pointers to search an element.

15. The roots of a quadratic equation of the form  $ax^2+bx+c = 0$  are given by the following equations:

$$X1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

$$X2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

Write a function to calculate the roots. The function must use two pointer parameters, one to receive the coefficients a, b and c and the other to send the roots to the calling function.

## ASSIGNMENT IV

1.
  - a) Write short notes on pointer to void.
  - b) Write short notes on Address Arithmetic.
  - c) Explain pointer to pointer with an example.
2.
  - a) How to use pointers as arguments in a function? Explain through an example.
  - b) Write a 'C' function using pointers to exchange the values stored in two locations in the memory.
3.
  - a) Explain how strings can be stored using a multidimensional arrays?
  - b) What are the string in-built functions available? Write in detail about each one of them with an example.
  - c) Write a C program to replace a particular word by another word in a given string.
4.
  - a) What is a pointer variable? How is a pointer variable different from an ordinary variable.
  - b) Distinguish between address operator and dereferencing operator.
  - c) Write a C Program to illustrate the use of indirection operator "\*" to access the value pointed by a pointer.
5. What is the purpose of array of pointers? illustrate with an example.
6.
  - a) Distinguish between array of pointers and pointer to array with examples.
  - b) Explain pointer to function and function returning pointer with example.